

SENAI FATESG

Análise e Desenvolvimento de Sistemas — 3º Semestre

Arquitetura e Projeto de Software

DOCUMENTO DE ARQUITETURA DE SOFTWARE

CarRepair

Sistema de Gestão de Oficina Mecânica

Versão 4.0

01 de abril de 2025

SENAI FATESG

Análise e Desenvolvimento de Sistemas — 3º Semestre
Arquitetura e Projeto de Software

Histórico de Versões

Versão	Data	Responsável	Descrição
v1.0	01/04/2025	Cassiano Abreu	Visão geral, stack e diagrama de camadas
v2.0	01/04/2025	Cassiano Abreu	Modelo de dados, UUID e persistência
v3.0	01/04/2025	Cassiano Abreu	DTOs, mappers, validação e contrato de erros
V4.0	25/05/2026	Cassiano Abreu	Domínios Mecânico e Usuário + frontend integrado com design + editar/excluir + toast

Colaboradores

Versão
Caio Nunes de Abreu
Cassiano Nunes de Abreu
Gabriel Naoki Uto Turigoe
Wyllian Mariano

1. Visão Geral do Sistema

O CarRepair é um sistema acadêmico para gestão de oficina mecânica, desenvolvido com backend em Java 21 + Spring Boot e frontend em Angular 21. O objetivo central é demonstrar, em um cenário realista e didático, a aplicação de arquitetura em camadas, separação por domínios, consumo de API REST e modelagem relacional em PostgreSQL.

O sistema organiza o ciclo completo de atendimento de uma oficina:

- Cadastro de clientes e seus veículos
- Cadastro de mecânicos responsáveis pela execução técnica
- Cadastro de usuários operadores do sistema
- Abertura e acompanhamento de Ordens de Serviço (OS)
- Registro de serviços executados e peças aplicadas em cada OS

1.1 Objetivos Acadêmicos

- Demonstrar evolução de código monolítico para arquitetura em camadas profissional
- Aplicar padrões de projeto: DTO, Mapper, Repository, Service, Validation, Controller
- Consumir API REST com tratamento centralizado de erros no frontend
- Modelar banco de dados relacional com PostgreSQL
- Praticar versionamento com Git Flow e Pull Requests

1.2 Escopo do Sistema

O sistema cobre os seguintes módulos funcionais:

- Gestão de Clientes
- Gestão de Veículos
- Gestão de Mecânicos
- Gestão de Usuários
- Gestão de Serviços
- Gestão de Ordens de Serviço

2. Stack Tecnológica

Camada	Tecnologia	Versão / Detalhe
Linguagem Backend	Java	21 (LTS)
Framework Backend	Spring Boot	Monolito modular
Persistência	Spring Data JPA + Hibernate	JpaRepository
Banco de dados	PostgreSQL	UUID, ENUM, TIMESTAMPTZ, pgcrypto
Linguagem Frontend	TypeScript	6.0.x
Framework Frontend	Angular	21 (next 22)
HTTP Client	Angular HttpClient + RxJS	7.8.x
Build Backend	Maven	Gestão de dependências
Build Frontend	Angular CLI	21
Redução de código	Lombok	Getters, Setters, Data
Testes Frontend	Vitest + JSDOM	-

3. Evolução Arquitetural — Versões

3.1 Versão 1.0 — Estrutura e Stack Base

Foco: Definição da pilha tecnológica e padrão de camadas

Data: Aula 01

Nesta versão foi estabelecida a arquitetura base do projeto. Foram definidas as tecnologias e a separação de responsabilidades por camada.

Decisões tomadas:

- Adoção de Java 21 com Spring Boot para o backend
- Escolha de arquitetura monolítica modular com separação em camadas
- Definição do fluxo: Controller → Service → Validation → Repository → Model
- Setup do Maven como gerenciador de dependências
- Definição do banco de dados como PostgreSQL

Diagrama de camadas definido na v1.0:



3.2 Versão 2.0 — Persistência e Padronização

Foco: Herança de infraestrutura, auditoria e Spring Data JPA

Data: Aula 02

Foco na padronização das entidades e da camada de persistência, reduzindo repetição de código e garantindo rastreabilidade no banco.

Implementações desta versão:

- Criação da classe BaseModel (@MappedSuperclass) com campos comuns a todas as entidades:
 - id (UUID gerado automaticamente)
 - ativo (boolean — suporte a soft delete)
 - dataHoraCriacao (Date — auditoria)
- Criação da classe BaseDTO espelhando os campos de BaseModel para a API
- Adoção de JpaRepository como base para todos os repositórios
- Configuração do banco PostgreSQL com uso de pgcrypto e UUID

3.3 Versão 3.0 — Reuso, Validação e Contrato de API

Foco: Generics, desacoplamento, DTOs/Mappers e tratamento de erros

Data: Aula 03

Introdução das classes genéricas do pacote core, eliminando duplicação de código entre os domínios e padronizando o contrato da API.

Implementações desta versão:

- GenericController — CRUD completo reutilizável por herança
- GenericService — orquestração genérica com hook beforeInsert
- GenericValidation — camada de validação reutilizável
- GenericMapper / IGenericMapper — contrato de conversão Model ↔ DTO
- IGenericRepository — interface base para todos os repositórios
- GlobalExceptionHandler (@RestControllerAdvice) — tratamento centralizado de exceções
- ErrorResponse, BusinessException, FieldValidationException, RuleValidationException — contrato de erros padronizado
- Domínios implementados: Cliente, Veículo, Serviço, Ordem de Serviço

No frontend angular, esta versão consolida:

- ApiBaseService — serviço HTTP base com get, post, put, delete
- HttpErrorInterceptor — interceptor global de erros HTTP
- MensagemService + MensagensComponent — exibição amigável de erros e notificações

3.4 Versão 4.0 — Expansão de Domínios

Foco: Implementação dos domínios Mecânico e Usuário no backend

O frontend já possuía páginas e services para os domínios Mecânico e Usuário, porém os endpoints correspondentes não existiam no backend. Esta versão corrige essa lacuna.

Domínio Mecânico — implementado:

- MecanicoModel — entidade JPA com campos: nome, cpf, email, telefone, especialidade, crea
- MecanicoDTO — objeto de transferência com os mesmos campos
- IMecanicoRepository / MecanicoRepository — camada de persistência
- IMecanicoService / MecanicoService — lógica de negócio
- IMecanicoValidation / MecanicoValidation — validações do domínio
- IMecanicoMapper / MecanicoMapper — conversão Model ↔ DTO
- MecanicoController — endpoint REST /mecanicos

Domínio Usuário — implementado:

- Mesma estrutura do domínio Mecânico, apontando para /usuarios
- Campos: nome, login, email, perfil (ENUM), senha_hash

Pendência relacionada – implementado:

- Após conclusão dos domínios Mecânico e Usuário, adicionar relacionamentos na OrdemServicoModel
- Campos a incluir: mecanicoResponsavelId, usuarioResponsavelId, servicosExecutados, pecasAplicadas

4. Arquitetura Atual (V4.0)

4.1 Visão das Camadas — Backend

Camada	Pacote / Pasta	Responsabilidade
Model	business.<dominio>.*Model	Entidade JPA — mapeia tabela no banco
DTO	business.<dominio>.*DTO	Objeto de transferência — contrato da API
Mapper	business.<dominio>.*Mapper	Conversão entre Model e DTO
Repository	business.<dominio>.*Repository	Persistência via Spring Data JPA
Validation	business.<dominio>.*Validation	Regras de negócio e validações
Service	business.<dominio>.*Service	Orquestração da lógica de negócio
Controller	business.<dominio>.*Controller	Endpoints REST — entrada e saída da API
Core (genéricos)	core.*	Classes base reutilizáveis por todos os domínios

4.2 Pacote Core — Classes Genéricas

O pacote core contém as classes base que eliminam duplicação de código entre os domínios:

Classe / Interface	Tipo	Função
BaseModel	Classe abstrata	Campos id, ativo, dataHoraCriacao para todas as entidades
BaseDTO	Classe abstrata	Campos id, active para todos os DTOs
GenericController	Classe abstrata	CRUD completo: GET, POST, PUT, DELETE
IGenericController	Interface	Contrato do controller genérico
GenericService	Classe abstrata	Orquestração com hook beforeInsert
IGenericService	Interface	Contrato do service genérico
GenericValidation	Classe abstrata	Validações reutilizáveis
IGenericValidation	Interface	Contrato da validation genérica
GenericMapper	Classe abstrata	Base para conversão Model ↔ DTO
IGenericMapper	Interface	Contrato do mapper genérico
IGenericRepository	Interface	Estende JpaRepository do Spring Data
GlobalExceptionHandler	Componente	Tratamento centralizado de exceções REST

SENAI FATESG

Análise e Desenvolvimento de Sistemas — 3º Semestre
Arquitetura e Projeto de Software

ErrorResponse	Classe	Estrutura padronizada de resposta de erro
CorsConfig	Configuração	Liberação de CORS para o frontend Angular

4.3 Domínios Implementados

Domínio	Endpoint	Status Backend	Status Frontend
Cliente	/clientes	Implementado	Implementado
Veículo	/veiculos	Implementado	Implementado
Serviço	/servicos	Implementado	Implementado
Ordem de Serviço	/ordens-servico	Implementado	Implementado
Mecânico	/mecanicos	Implementado (v4.0)	Implementado
Usuário	/usuarios	Implementado (V4.0)	Implementado

4.4 Fluxo de uma Requisição

Toda requisição ao backend percorre obrigatoriamente o seguinte fluxo:

```
HTTP Request → Controller → Service → Validation → Repository → Banco de Dados
HTTP Response ← Controller ← Mapper (Model → DTO) ← Service ← Repository
```


5. Arquitetura do Frontend (Angular)

5.1 Estrutura de Pastas

```
src/app/
  core/
    http/           ← ApiService + HttpErrorInterceptor
    validacoes/     ← Validações reutilizáveis
  modelos/         ← Interfaces TypeScript por domínio
  paginas/         ← Componentes de tela (standalone)
    clientes/
    veiculos/
    mecanicos/
    ordens-servico/
    dashboard/
  services/dominios/ ← Services REST por domínio
  shared/mensagens/ ← MensagemService + MensagensComponent
  app.routes.ts     ← Rotas da aplicação
```

5.2 Rotas da Aplicação

Rota	Componente	Descrição
/dashboard	DashboardComponent	Tela inicial do sistema
/clientes	ClientesComponent	Cadastro e listagem de clientes
/veiculos	VeiculosComponent	Cadastro e listagem de veículos
/mecanicos	MecanicosComponent	Cadastro e listagem de mecânicos
/ordens-servico	OrdensServicoComponent	Cadastro e acompanhamento de OS

5.3 Tratamento de Erros

O frontend adota uma diretriz clara: nenhum dado simulado (mock) deve mascarar falhas de API. O fluxo de erro é:

- Requisição HTTP falha
- HttpErrorInterceptor captura o erro
- Traduz o código HTTP em mensagem amigável
- MensagemService envia a mensagem para a UI
- MensagensComponent exibe a notificação ao usuário

Mapeamento de erros:

Status HTTP	Mensagem exibida
0	Falha de rede — verifique sua conexão
400	Dados inválidos — usa mensagem do backend

SENAI FATESG

Análise e Desenvolvimento de Sistemas — 3º Semestre

Arquitetura e Projeto de Software

401	Sessão expirada
403	Acesso negado
404	Recurso não encontrado
5xx	Erro interno do servidor

6. Modelo de Banco de Dados

6.1 Tecnologia e Recursos

- PostgreSQL com extensão pgcrypto
- Chave primária UUID (gen_random_uuid())
- ENUM para status da Ordem de Serviço
- TIMESTAMPTZ para campos de auditoria temporal
- Índices nos campos de relacionamento

6.2 Tabelas

Tabela	Descrição
usuarios	Operadores do sistema (login, perfil, senha_hash)
clientes	Proprietários dos veículos
veiculos	Automóveis vinculados a clientes
mecanicos	Profissionais técnicos da oficina
ordens_servico	Tabela central — vincula cliente, veículo, usuário e mecânico
ordem_servico_servicos	Itens de mão de obra executados na OS
ordem_servico_pecas_aplicadas	Itens de peças utilizadas na OS

6.3 Relacionamentos

- 1:N — clientes → veiculos
- 1:N — clientes → ordens_servico
- 1:N — usuarios → ordens_servico
- 1:N — mecanicos → ordens_servico
- 1:N — veiculos → ordens_servico
- 1:N — ordens_servico → ordem_servico_servicos
- 1:N — ordens_servico → ordem_servico_pecas_aplicadas

6.4 ENUM de Status da OS

```
CREATE TYPE status_ordem_servico AS ENUM (  
    'aberta',  
    'em_execucao',  
    'finalizada',  
    'cancelada'  
);
```

6.5 Índices

```
CREATE INDEX idx_veiculos_cliente ON veiculos(cliente_id);  
CREATE INDEX idx_ordens_cliente ON ordens_servico(cliente_id);  
CREATE INDEX idx_ordens_veiculo ON ordens_servico(veiculo_id);  
CREATE INDEX idx_servicos_ordem ON  
ordem_servico_servicos(ordem_servico_id);  
CREATE INDEX idx_pecas_ordem ON  
ordem_servico_pecas_aplicadas(ordem_servico_id);
```

7. Pendências e Próximos Passos

7.1 Pendências Críticas

Item	Responsável	Status	Descrição
1	Caio	Concluído	Incompatibilidade de paginação resolvida — frontend adaptado
2	Cassiano	Concluído	Implementar domínio Mecânico no backend
4	Wyllian	Concluído	Alinhamento da OrdemServico concluído — campos implementados e integrados ao frontend

7.2 Melhorias Futuras

- Autenticação real com JWT
- Guards de rota no Angular
- Paginação e filtros nas listagens
- CRUD completo (edição e exclusão) para todos os domínios
- Testes unitários por serviço e componente
- Módulo de relatórios e indicadores
- Catálogo de peças e estoque
- Histórico de manutenção por veículo

8. ADR's

ADR-001

Definição da Stack e Arquitetura Base

Status: **Em vigor** Data: Aula 01 Autor: Equipe

Caio Abreu | Cassiano Abreu | Gabriel Naoki | Wyllian Mariano
SENAI FATESG — Goiânia, GO — 01 de abril de 2025

O projeto CarRepair precisava de uma base tecnológica sólida para suportar o desenvolvimento de um sistema de gestão de oficina mecânica com alta manutenibilidade e organização didática. Era necessário definir a linguagem, o framework, o banco de dados e o padrão de organização do código antes de iniciar qualquer implementação. Sem essas definições, cada integrante poderia adotar abordagens diferentes, gerando inconsistências e dificuldade de integração.

Decisão

Utilizar Java 21 com Spring Boot no backend, adotando arquitetura de monolito modular com separação obrigatória em camadas: Model, Repository, Validation, Service e Controller. No frontend, adotar Angular 21 com TypeScript. Banco de dados PostgreSQL.

Fluxo obrigatório de todas as requisições:

Controller → Service → Validation → Repository → Model

Justificativa

- Java 21 (LTS) traz recursos modernos como Virtual Threads e Pattern Matching, aumentando produtividade e desempenho
- Spring Boot acelera o setup e elimina configuração manual de infraestrutura
- A separação em camadas isola responsabilidades, facilita testes unitários e torna o código mais compreensível para fins acadêmicos
- O monolito modular é adequado para o escopo acadêmico — evita complexidade desnecessária de microsserviços
- Angular 21 com TypeScript garante tipagem estática, organização por componentes e alinhamento com o mercado

- PostgreSQL é robusto, open source, suporta UUID nativo e é amplamente utilizado no mercado

Alternativas Consideradas

- Microsserviços — descartado por adicionar complexidade desnecessária ao escopo acadêmico
- Quarkus ou Micronaut — descartados pela menor familiaridade da equipe com esses frameworks
- MySQL no lugar de PostgreSQL — descartado pela preferência por UUID nativo e recursos avançados do PostgreSQL
- React ou Vue no frontend — descartados em favor do Angular pela estrutura mais rígida e didaticamente mais próxima do padrão enterprise

Consequências

- Todo novo domínio deve obrigatoriamente seguir o fluxo Controller → Service → Validation → Repository → Model
- A separação de responsabilidades aumenta o número de arquivos por domínio, mas garante manutenibilidade
- O backend escalável permite evoluir para microsserviços futuramente sem reescrita total
- A equipe precisa manter disciplina na criação e organização das classes por camada

ADR-002

BaseModel e Spring Data JPA

Status: Em vigor **Data:** Aula 02 **Autor:** Equipe

Caio Abreu | Cassiano Abreu | Gabriel Naoki | Wyllian Mariano

SENAI FATESG — Goiânia, GO — 01 de abril de 2025

Todas as entidades do sistema precisavam de campos comuns de auditoria: identificador único, data de criação e indicador de status ativo/inativo. Sem uma estratégia centralizada, esses campos seriam duplicados manualmente em cada entidade, gerando retrabalho, inconsistências e violando o princípio DRY (Don't Repeat Yourself). Além disso, era necessário definir como o acesso ao banco de dados seria abstraído para evitar SQL manual disperso pelo código.

Decisão

Criar uma classe abstrata `BaseModel` anotada com `@MappedSuperclass` contendo os campos comuns a todas as entidades: `id` (UUID gerado automaticamente), `ativo` (boolean para soft delete) e `dataHoraCriacao` (Date para auditoria). Criar também uma classe `BaseDTO` espelhando os campos `id` e `ativo` para o contrato da API. Adotar `JpaRepository` do Spring Data como base para todos os repositórios, abstraindo o SQL manual.

Estrutura do `BaseModel`:

```
@MappedSuperclass
public abstract class BaseModel {
    @Id @GeneratedValue
    private UUID id;
    private boolean ativo;
    private Date dataHoraCriacao;
}
```

Justificativa

- `BaseModel` elimina duplicação de campos de auditoria — princípio DRY aplicado na camada de entidades
- UUID como chave primária evita colisões em ambientes distribuídos e é mais seguro que IDs sequenciais expostos na API
- O campo `ativo` viabiliza soft delete — o registro nunca é removido fisicamente, preservando histórico e integridade referencial
- `JpaRepository` abstrai completamente o SQL manual, permitindo foco total na lógica de negócio
- Spring Data gera automaticamente queries por convenção de nomes (ex: `findById`, `existsByCpf`, `findByAtivo`)
- `BaseDTO` desacopla o contrato da API da estrutura interna do banco de dados

Alternativas Consideradas

- Herança com `SINGLE_TABLE` (`@Inheritance`) — descartado por misturar dados de domínios distintos em uma única tabela
- JDBC puro sem JPA — descartado pela verbosidade e necessidade de SQL manual para operações simples
- IDs sequenciais (`Long auto_increment`) — descartado em favor de UUID para maior segurança e escalabilidade
- Campos de auditoria duplicados em cada entidade — descartado por violar DRY e gerar retrabalho

Consequências

- Todas as entidades herdam automaticamente `id`, `ativo` e `dataHoraCriacao` sem nenhuma repetição de código

- Soft delete implementado via campo ativo — exclusões são lógicas, não físicas, preservando histórico
- A dependência do Spring Data JPA impõe uso obrigatório de uma implementação JPA (Hibernate por padrão)
- Migrações de schema precisam ser gerenciadas manualmente ou via ferramenta (ex: Flyway) em versões futuras
- Qualquer novo domínio herda automaticamente o padrão de auditoria sem esforço adicional

ADR-003

Generics, Mappers e Tratamento Global de Exceções

Status: **Em vigor** Data: Aula 03 Autor: Equipe

Caio Abreu | Cassiano Abreu | Gabriel Naoki | Wyllian Mariano

SENAI FATESG — Goiânia, GO — 01 de abril de 2025

Com múltiplos domínios implementados (Cliente, Veículo, Serviço, Ordem de Serviço), ficou evidente que o CRUD básico era idêntico em todos eles — cada domínio reescrevia as mesmas operações de listar, buscar por ID, criar, atualizar e deletar no Controller e no Service. Além disso, a API não possuía um contrato padronizado para respostas de erro, retornando exceções brutas do Spring ao frontend Angular, dificultando o tratamento no lado do cliente.

Decisão

Introduzir classes genéricas no pacote core usando Java Generics para parametrizar por tipo de entidade, eliminando a duplicação de CRUD entre os domínios. Criar um `GlobalExceptionHandler` com `@RestControllerAdvice` para tratamento centralizado de todas as exceções. Adotar DTOs obrigatoriamente em todas as respostas da API, com Mappers dedicados para conversão entre Model e DTO.

Classes genéricas criadas:

<code>GenericController<M, D, S, MP></code>	→	CRUD completo por herança
<code>GenericService<M, R, V></code>	→	Orquestração com hook <code>beforeInsert</code>
<code>GenericValidation<M></code>	→	Validações reutilizáveis
<code>GenericMapper<M, D></code>	→	Conversão Model <-> DTO
<code>IGenericRepository<M></code>	→	Estende <code>JpaRepository</code>

Justificativa

- Generics eliminam duplicação de código CRUD — cada domínio herda o comportamento sem reescrever nada
- DTOs desacoplam o contrato da API da estrutura interna das entidades JPA — mudanças no banco não quebram o frontend
- O `GlobalExceptionHandler` garante que todos os erros retornem no mesmo formato (`ErrorResponse`), facilitando o tratamento no Angular
- A camada de Validation separada do Service segue o princípio de responsabilidade única (SRP)
- O hook `beforeInsert` no `GenericService` permite customização por domínio sem quebrar o fluxo genérico
- Interfaces para cada camada (`IGenericService`, `IGenericMapper`, etc.) garantem desacoplamento e facilitam testes

Alternativas Consideradas

- Copiar e colar o CRUD em cada domínio — descartado por gerar retrabalho, inconsistências e dificultar manutenção
- `MapStruct` ou `ModelMapper` para mapeamento — descartado para manter o projeto simples e didático, com mapeamento explícito
- Retornar entidades JPA diretamente na API — descartado por expor estrutura interna e criar acoplamento forte com o banco
- Tratamento de erro individual por controller — descartado por gerar duplicação e inconsistência nas respostas

Consequências

- Novos domínios ficam muito mais simples de implementar — apenas herdam as classes genéricas e sobrescrevem o necessário
- O contrato de erro padronizado (`ErrorResponse`) estabiliza a integração com o frontend Angular
- Aumenta o número de classes por domínio (interface + implementação para cada camada), exigindo disciplina da equipe
- A camada genérica cobre 80% dos casos — casos especiais ainda precisam de implementação específica por domínio
- Nunca expor entidades JPA diretamente nas respostas da API — regra obrigatória do projeto a partir desta versão

ADR-004

Implementação dos Domínios Mecânico e Usuário Backend (Spring Boot) + Frontend (Angular)

SENAI FATESG

Análise e Desenvolvimento de Sistemas — 3º Semestre
Arquitetura e Projeto de Software

Status: Em vigor **Data:** 01/04/2025 **Autor:** Cassiano Abreu

Caio Abreu | Cassiano Abreu | Gabriel Naoki | Wyllian Mariano
SENAI FATESG — Goiânia, GO — 01 de abril de 2025

O frontend Angular já possuía páginas standalone e services configurados para os domínios Mecânico (/mecanicos) e Usuário (/usuarios), porém esses endpoints não existiam no backend Spring Boot. Isso causava erros 404 capturados pelo `HttpErrorInterceptor` em tempo de execução, impedindo o funcionamento das telas de mecânicos e bloqueando a criação da tela de usuários.

Adicionalmente, a ausência desses domínios bloqueava a atividade 4, que depende de `mecanicoResponsavelId` e `usuarioResponsavelId` para alinhar os campos da `OrdemServico` entre frontend e backend.

Decisão

Implementar os domínios Mecânico e Usuário de forma completa tanto no backend quanto no frontend, seguindo rigorosamente os padrões já estabelecidos nas versões anteriores.

No backend: seguir o padrão genérico do ADR-003 (Model, DTO, IRepository, Repository, IService, Service, IValidation, Validation, IMapper, Mapper, Controller) em pacotes isolados dentro de `business.*`.

No frontend: seguir o padrão dos domínios existentes (interface TypeScript, service herdando `ApiBaseService`, componente standalone com formulário e listagem, rota registrada no `app.routes.ts`).

Todas as alterações organizadas em branch separado (`feat/dominio-mecanico`) com commits atômicos por responsabilidade, submetidos via Pull Request.

SENAI FATESG

Análise e Desenvolvimento de Sistemas — 3º Semestre

Arquitetura e Projeto de Software

Domínio Mecânico — br.edu.senai.fatesg.ads3.car_repair.business.mecanicos

Arquivo	Camada	Responsabilidade
MecanicoModel.java	Model / Entidade JPA	Tabela Mecanico — campos: nome, cpf, email, telefone, especialidade, crea
MecanicoDTO.java	DTO	Contrato da API — mesmos campos do Model
IMecanicoRepository.java	Interface Repository	Estende IGenericRepository<MecanicoModel>
MecanicoRepository.java	Repository	Classe vazia — implementação pelo Spring Data
IMecanicoService.java	Interface Service	Estende IGenericService<MecanicoModel>
MecanicoService.java	Service	beforeInsert: define ativo=true e dataHoraCriacao
IMecanicoValidation.java	Interface Validation	Estende IGenericValidation<MecanicoModel>
MecanicoValidation.java	Validation	Implementação das validações do domínio
IMecanicoMapper.java	Interface Mapper	Estende IGenericMapper<MecanicoModel, MecanicoDTO>
MecanicoMapper.java	Mapper	Conversão bidirecional Model <-> DTO
MecanicoController.java	Controller	Endpoint REST /mecanicos — CRUD completo por herança

Domínio Usuário — br.edu.senai.fatesg.ads3.car_repair.business.usuarios

Arquivo	Camada	Responsabilidade
UsuarioModel.java	Model / Entidade JPA	Tabela Usuario — campos: nome, login, email, perfil, senha_hash
UsuarioDTO.java	DTO	Contrato da API — expõe nome, login, email, perfil (sem senha_hash)
IUsuarioRepository.java	Interface Repository	Estende IGenericRepository<UsuarioModel>
UsuarioRepository.java	Repository	Classe vazia — implementação pelo Spring Data
IUsuarioService.java	Interface Service	Estende IGenericService<UsuarioModel>

SENAI FATESG

Análise e Desenvolvimento de Sistemas — 3º Semestre Arquitetura e Projeto de Software

UsuarioService.java	Service	beforeInsert: define ativo=true e dataHoraCriacao
IUsuarioValidation.java	Interface Validation	Estende IGenericValidation<UsuarioModel>
UsuarioValidation.java	Validation	Implementação das validações do domínio
IUsuarioMapper.java	Interface Mapper	Estende IGenericMapper<UsuarioModel, UsuarioDTO>
UsuarioMapper.java	Mapper	Conversão bidirecional Model <-> DTO — senha_hash não exposta no DTO
UsuarioController.java	Controller	Endpoint REST /usuarios — CRUD completo por herança

Domínio Mecânico — já existia, integração ativada

O componente de mecânicos já estava implementado no projeto. Com a criação do endpoint /mecanicos no backend, a integração passou a funcionar sem nenhuma alteração no frontend.

- MecanicosComponent — formulário com nome, especialidade e telefone; listagem em tabela
- MecanicosService — herda ApiService; métodos listar() e adicionar()
- Mecanico (interface) — campos: id, nome, especialidade, telefone
- Rota /mecanicos — já registrada no app.routes.ts

Domínio Usuário — criado do zero

O domínio Usuário foi integralmente implementado no frontend seguindo o mesmo padrão dos demais domínios:

Arquivo	Tipo	Descrição
usuario.ts	Interface TypeScript	Campos: id, nome, login, email, perfil (union type)
usuarios.service.ts	Service Angular	Herda ApiService — métodos listar() e adicionar()
usuarios.component.ts	Componente	Formulário com validação, listagem e integração com service
usuarios.component.html	Template	Formulário com select de perfil (administrativo/atendente/gestor)

SENAI FATESG

Análise e Desenvolvimento de Sistemas — 3º Semestre

Arquitetura e Projeto de Software

<code>usuarios.component.css</code>	Estilo	Estilização dos erros de validação
-------------------------------------	--------	------------------------------------

Alterações nos arquivos globais do Angular:

- `app.routes.ts` — adicionada rota `/usuarios` apontando para `UsuariosComponent`
- `app.html` — adicionado link de navegação para `/usuarios` no menu principal

Justificativa

- Seguir o padrão genérico do ADR-003 garante consistência com os demais domínios e minimiza risco de erros
- Branches separados e commits atômicos por responsabilidade garantem rastreabilidade das mudanças
- A senha_hash não é exposta no `UsuarioDTO` por segurança — separação entre Model (banco) e DTO (API) aplicada
- O campo perfil usa union type no TypeScript alinhado com o ENUM do PostgreSQL
- A implementação frontend completa garante que o domínio Usuário funcione de ponta a ponta
- Nenhum dado simulado (mock) foi utilizado — a diretriz do projeto foi seguida rigorosamente

Alternativas Consideradas

- Criar endpoints sem as camadas completas (apenas Controller) — descartado por quebrar o padrão arquitetural do ADR-003
- Usar mock/stub no frontend enquanto o backend não estava pronto — descartado pela diretriz oficial do projeto
- Implementar autenticação JWT completa nesta versão — adiado para manter o escopo da atividade controlado
- ENUM Java para o campo perfil no Model — adiado para uma versão futura; String foi adotado por simplicidade

Consequências

- Os endpoints `/mecanicos` e `/usuarios` estão disponíveis no backend, resolvendo os erros 404 do frontend
- A tela de Usuários está funcional de ponta a ponta — formulário, validação, listagem e integração com a API
- A atividade 4 (alinhamento da `OrdemServico` com `mecanicoResponsavelId` e `usuarioResponsavelId`) fica desbloqueada
- 22 arquivos adicionados ao backend + 7 ao frontend — todos seguindo os padrões estabelecidos nos ADRs anteriores
- O domínio Usuário ainda não possui autenticação real — a senha_hash é persistida mas o mecanismo de login será implementado em versão futura

- O campo perfil do Usuário é String no backend — uma evolução futura pode convertê-lo para ENUM Java alinhado com o tipo PostgreSQL

Atualização v4.0 — Integração Angular Completa

Contexto

Com os domínios Mecânico e Usuário funcionando no backend (v4.0), o frontend Angular precisava ser elevado ao mesmo nível de qualidade: o design estava genérico, sem aderência ao HTML estático aprovado pela equipe, e as operações de edição e exclusão não estavam implementadas em nenhum domínio. Além disso, o feedback visual ao usuário (mensagens de sucesso/erro) era esteticamente inadequado.

Decisão

Integrar o design do HTML estático aprovado diretamente nos componentes Angular existentes, implementar operações de edição (PUT) e exclusão (DELETE) em todos os domínios, e substituir o sistema de mensagens por um toast flutuante com estética consistente. Todas as alterações organizadas em duas branches separadas com Pull Requests.

Implementação — Frontend Angular V4.0

Design Integrado (branch: feat/frontend-carrepair-ui-v2)

O design do HTML estático aprovado foi transposto integralmente para os componentes Angular. CSS variables globais (--p1, --accent, --bg, --surface) definidas em styles.css. Navbar com fundo azul escuro (#1a2744) e logo com destaque dourado. Hero com mock do sistema na página inicial. Cada página interna possui header escuro, botão dourado para novo registro, tabela estilizada com badges de status e formulário colapsável com animação fadeDown. Todos os 6 domínios (clientes, veículos, mecânicos, serviços, ordens de serviço, usuários) consomem a API real na porta 9081 — nenhum dado simulado. Validação de CPF com algoritmo Luhn idêntico ao backend, e mensagens de erro do GlobalExceptionHandler exibidas ao usuário.

Editar e Excluir + Toast (branch: feat/frontend-edit-delete-toast)

Botões Editar e Excluir adicionados na coluna Ações de todas as tabelas. Editar preenche o formulário com os dados do registro e executa PUT `/{{endpoint}}/{{id}}` ao salvar. Excluir solicita confirmação via dialog nativo e executa DELETE `/{{endpoint}}/{{id}}`. O `ApiBaseService` foi corrigido para aceitar `ResponseEntity<String>` do backend (responseType: 'text'). O `MensagensComponent` foi reescrito como toast flutuante: position fixed no canto superior direito, borda colorida por tipo (verde/vermelho), animação toastIn, ícone ✓/✗ em círculo. Correção da tela de Ordens de Serviço: nomes de cliente, veículo e mecânico resolvidos via forkJoin, tabela com overflow-x para caber na tela.

Justificativa

O CRUD completo (criar, listar, editar, excluir) é requisito fundamental de um sistema de gestão — a ausência de editar/excluir deixava o sistema incompleto para uso real. A integração do design aprovado garante consistência visual entre a documentação e o produto entregue. O toast substitui a barra de notificação invasiva por um componente

não-bloqueante, seguindo padrões modernos de UX. Branches separadas por responsabilidade mantiveram a rastreabilidade das mudanças.

Alternativas Consideradas

Reescrever o frontend do zero — descartado por desperdiçar o trabalho já existente nos services e rotas. Usar biblioteca de UI externa (Angular Material, PrimeNG) — descartado para manter controle total sobre o visual e evitar dependências desnecessárias. Modal customizado de confirmação — adiado em favor do dialog nativo do browser para simplicidade. Soft delete via campo ativo=false no PUT — descartado; o DELETE do GenericController já implementa exclusão lógica no backend.

Consequências

O frontend está completo e funcional de ponta a ponta — todos os 6 domínios com CRUD completo integrado à API real. O design é fiel ao HTML estático aprovado. Toast notifications melhoram a experiência do usuário sem bloquear a interface. Dois PRs abertos (feat/frontend-carrepair-ui-v2 e feat/frontend-edit-delete-toast) aguardam merge pelo Caio. A próxima evolução natural é a implementação de autenticação JWT e paginação com filtros nas listagens.

9. Equipe

Integrante
Caio Abreu
Cassiano Abreu
Gabriel Naoki
Wyllian Mariano